

Quanta Scout OS — Architecture & Technical Reference

Version: 2.0 · May 2026 Stack: Next.js 14 · Supabase · Groq · Resend · Vercel

Table of Contents

- 1. [What the System Does](#)
- 2. [Full Architecture Diagram](#)
- 3. [Layer-by-Layer Breakdown](#)
- 4. [Agent Design — How Every Agent Works](#)
- 5. [OpenClaw — The Messaging Gateway](#)
- 6. [Telegram Bot as the Interaction Layer](#)
- 7. [Current Tools & Free-Tier Constraints](#)
- 8. [Production Upgrades — What Changes and Why](#)
- 9. [AI Model Choices — Groq vs OpenAI](#)
- 10. [Deal State Machine](#)
- 11. [Limitations & Known Constraints](#)
- 12. [Future Scope](#)

1. What the System Does

Quanta Scout OS is an **AI-native venture scout intelligence platform**. It solves a specific operational problem: scouts discover startups in conversations, at events, and in WhatsApp groups — but investment teams need structured, comparable data. The gap between "how scouts naturally share" and "what investors need to receive" is where this system lives.

The Two Experiences

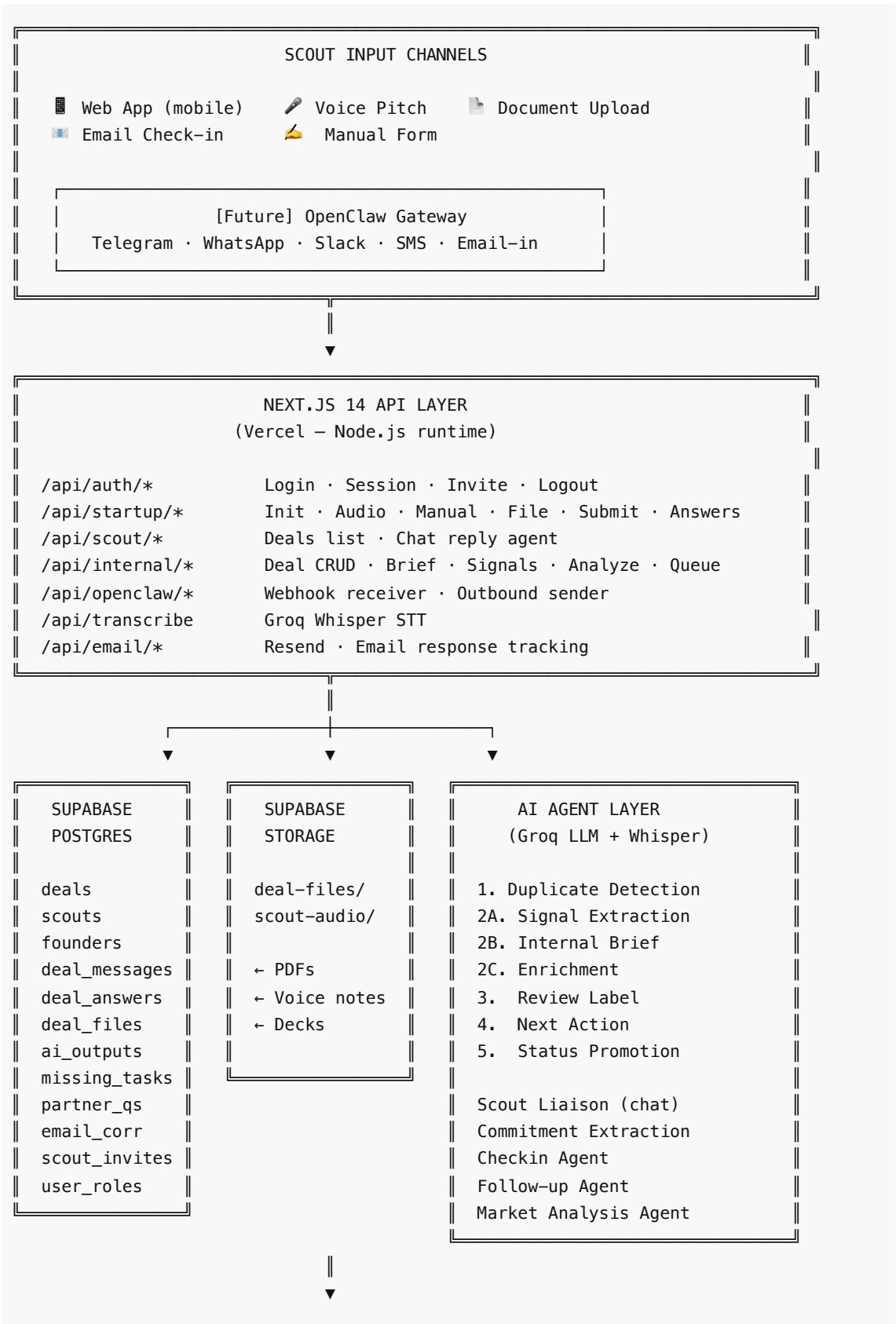
Scout Experience (mobile-first) Scouts interact the same way they'd text a colleague. They record a voice pitch in the car, type a quick message about a founder they met, or upload a pitch deck PDF. The system handles all structuring, follow-up, and routing. Scouts never navigate a CRM, fill out a form, or remember what they promised to send.

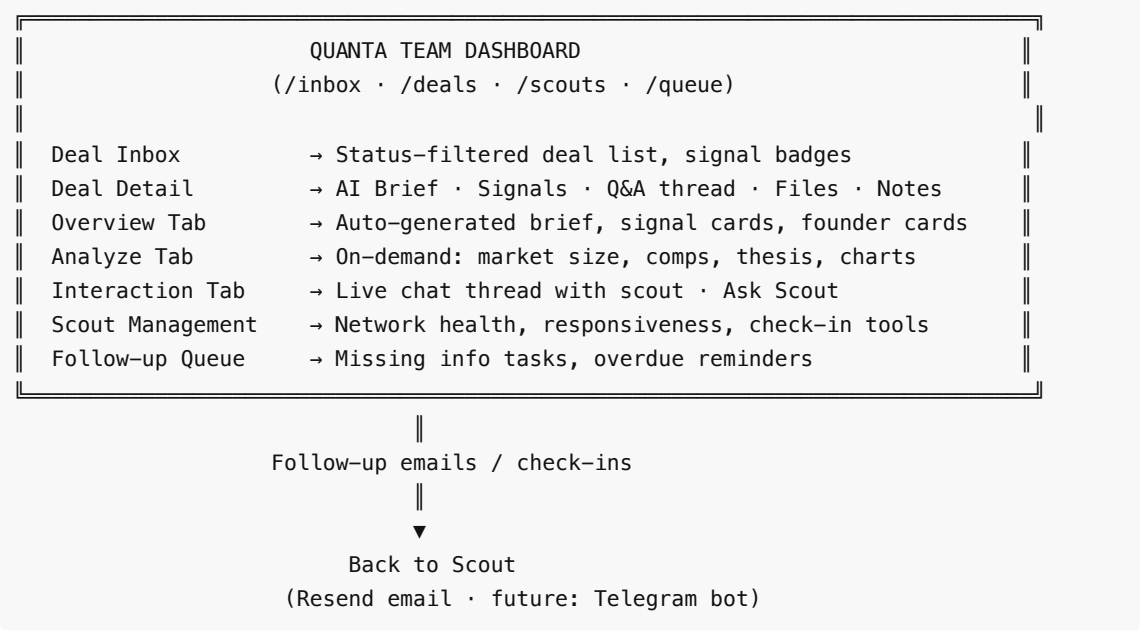
Quanta Team Experience (desktop-first) Every deal arrives pre-structured with qualitative VC signals, an investor-ready brief, open questions surfaced, and a direct line to the scout. Triage takes seconds. Deep research (market analysis, comparable companies, investment thesis) is one click away.

The Core Design Principles

Principle	Implementation
Scouts feel like they're texting	Conversational input, one question at a time, no forms
No numeric scores	Signals use strong / medium / early / weak / unclear — never percentages
Incomplete is fine	Deals are created with partial data; system collects the rest over time
Background intelligence	All AI processing is non-blocking; neither side waits for agents
Full auditability	Every AI output is logged with its input snapshot in <code>ai_outputs</code>

2. Full Architecture Diagram





3. Layer-by-Layer Breakdown

3.1 Scout Input Layer

The scout can submit a deal in three ways:

Voice Pitch The scout records audio directly in the browser. The audio blob is sent to `/api/startup/:id/audio` . The server transcribes it via Groq Whisper (whisper-large-v3-turbo), then simultaneously runs:

- Extraction: pulls startup_name, founder_names, one_line_description, category, stage, fundraising, traction, company_strength, scout_conviction
- Commitment detection: identifies promises ("I'll send the deck by Thursday")
- Distribution: maps the transcript to every Q&A answer field so they come pre-filled

Manual Entry Scout fills a short form. Sent to `/api/startup/:id/manual` . Same extraction pipeline runs to structure the data.

Document Upload Scout uploads a PDF, DOCX, or presentation. Stored in Supabase Storage. Text extracted, then enrichment agent reads it and populates deal fields and missing_info_tasks.

3.2 API Layer (Next.js 14 Route Handlers)

All API routes live in `src/app/api/` . Key route groups:

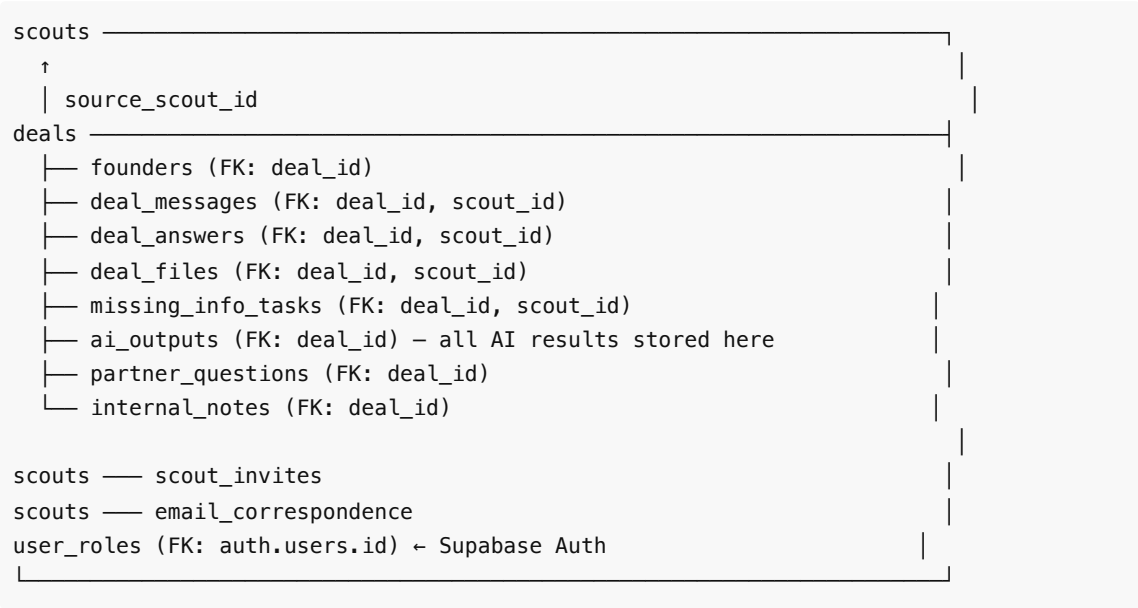
Group	What it handles
<code>/api/auth/*</code>	Login, logout, session, invite, complete-signup
<code>/api/startup/:id/*</code>	Deal lifecycle: init, audio, manual, file, answers, submit, discard
<code>/api/scout/deals/*</code>	Scout's deal list + chat reply endpoint
<code>/api/internal/*</code>	Team-facing: deal CRUD, brief gen, signals, analyze, queue, scheduler

/api/openclaw/*	Inbound webhook + outbound message sender
/api/transcribe	Groq Whisper speech-to-text
/api/email/*	Resend delivery + email response tracking

Every route has `export const dynamic = "force-dynamic"` to prevent Next.js Data Cache from serving stale responses. The deal messages endpoint additionally uses a direct `fetch()` with `cache: "no-store"` to bypass any SDK-level caching.

3.3 Database Layer (Supabase Postgres)

Core tables and relationships:



Row-level security is enabled on `user_roles`. API routes use the Supabase service role key (bypasses RLS) for all server-side operations. Middleware uses the anon key with the user's JWT session for routing decisions.

3.4 Auth Layer

Two roles exist:

Role	Access	Session
quanta	Full dashboard — all deals, all scouts	Supabase Auth + quanta_role cookie
scout	Own deals only, scout portal	Supabase Auth + quanta_role + quanta_scout_id cookies

The login API sets a `quanta_role` cookie on the response. The middleware reads this cookie for routing decisions (fast path — no DB query). If the cookie is missing, it falls back to querying `user_roles` via the Supabase session.

3.5 Storage Layer (Supabase Storage)

Two buckets:

Bucket	Contents	Access
deal-files	PDFs, pitch decks, documents	Private — signed URLs for download
scout-audio	Voice pitch recordings	Private — signed URLs for playback

Files are stored at path `{deal_id}/{timestamp}_{filename}` . Signed URLs are generated on demand via `/api/storage/signed-url` .

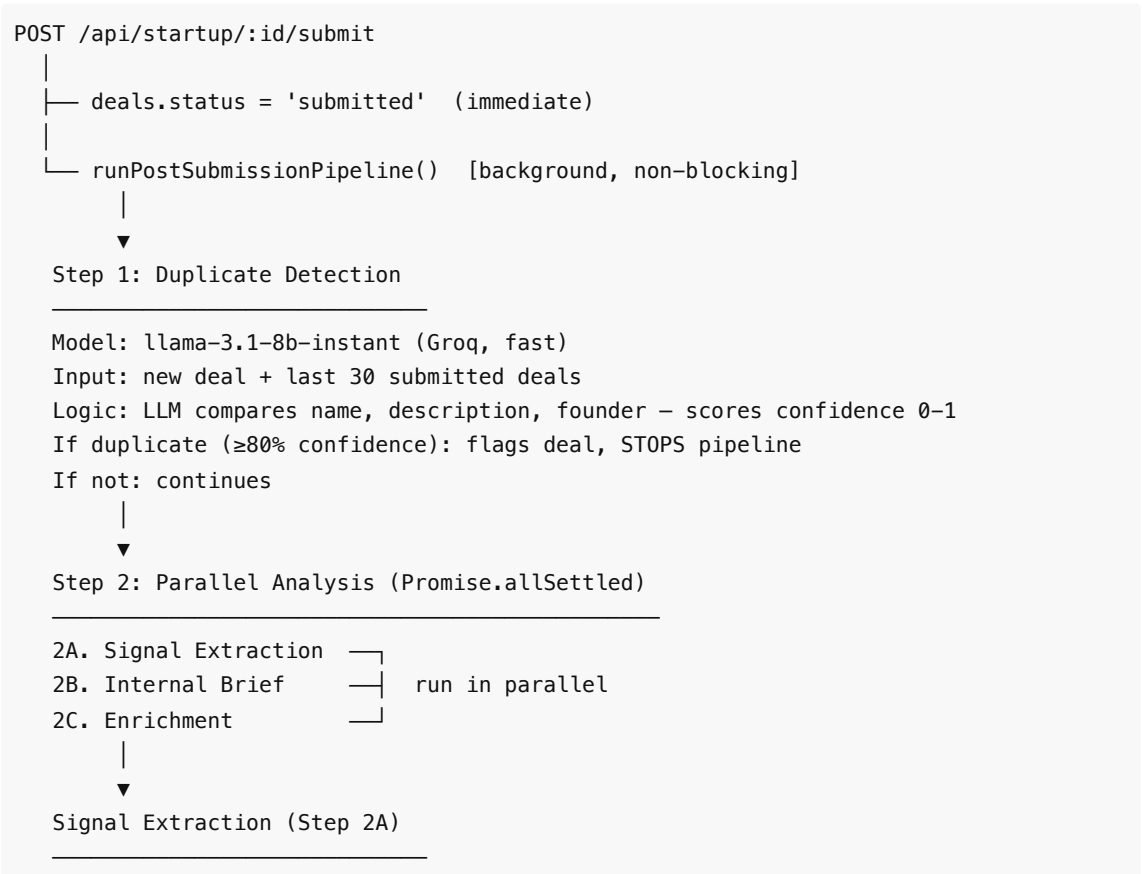
4. Agent Design — How Every Agent Works

All agents are **pure async TypeScript functions** in `src/agents/` . They:

- Read context from Supabase
- Call an LLM with a structured prompt
- Write results back to Supabase
- Return a typed result object
- Are wrapped in try/catch — one agent failing never blocks another

4.1 Post-Submission Pipeline

Triggered automatically after a scout submits. Runs in the background — the scout sees "Submitted" immediately.



Model: llama-3.3-70b-versatile (Groq, high quality)
Input: deal context + founders + Q&A + files + messages
Output: 4 signal cards (founder / market / traction / scout_conviction)
Each card has: level (strong/medium/early/weak/unclear) + evidence text
Plus: risk_flags[] array
Saved to: ai_outputs (output_type = 'signal_summary')

Internal Brief (Step 2B)

Model: llama-3.3-70b-versatile (Groq)
Input: same deal context as signals
Output: {
 brief_title, what_it_does, why_it_may_matter,
 known_facts[], open_questions[], suggested_next_action
}
Saved to: ai_outputs (output_type = 'internal_brief')
Side effect: open_questions are seeded as missing_info_tasks

Enrichment (Step 2C)

Model: llama-3.3-70b-versatile (Groq)
Input: extracted text from uploaded documents
Output: structured fields + missing_diligence_questions[]
Saved to: ai_outputs (output_type = 'enrichment')
Side effect: missing_diligence_questions become missing_info_tasks

|



Step 3: Review Label

Model: llama-3.1-8b-instant (Groq, fast)
Input: signal cards from Step 2A
Output: 'strong_candidate' | 'worth_exploring' | 'needs_more_info'
Written directly to: deals.review_label

Step 4: Next Action

Model: llama-3.1-8b-instant (Groq)
Input: brief + signals + pending tasks
Output: single specific action string
Saved to: ai_outputs (output_type = 'followup_question')

Step 5: Status Promotion

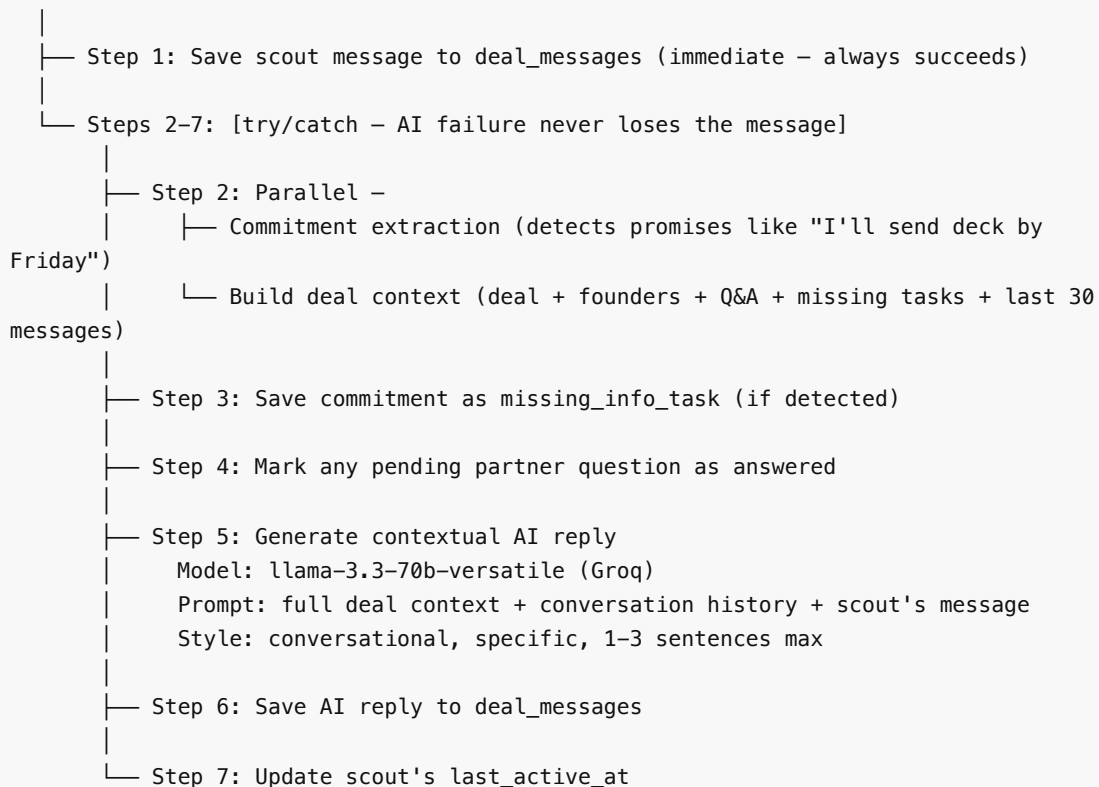
Logic: if missing_info_tasks exist → 'needs_info'
if no missing tasks → 'under_review'
Written to: deals.status

4.2 Scout Liaison Agent (Chat Reply)

Lives at `src/app/api/scout/deals/[id]/reply/route.ts` .

Every time a scout types a message in the Interaction tab:

Scout sends message

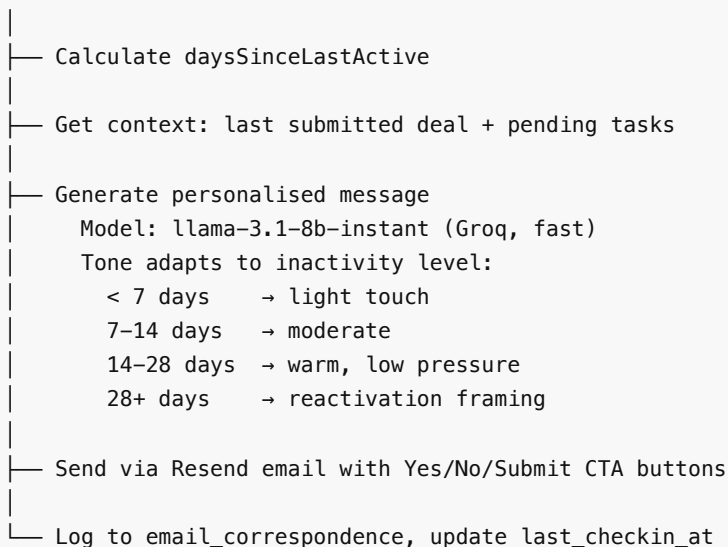


If the AI step throws (rate limit, timeout, etc.), the fallback reply "Thanks — we'll take a look and follow up if we have questions." is saved, and the route still returns 200 to the client.

4.3 Weekly Check-in Agent

Lives at `src/agents/checkin/index.ts` . Triggered by cron (Saturdays 9am UTC) or manually.

For each active scout with email:



4.4 Follow-up Agent

Lives at `src/agents/followup/index.ts` . Triggered by the scheduler or "Ask now" button.

```
For a given missing_info_task:
|
|— Resolve scout email (via task.scout_id or deal.source_scout_id fallback)
|
|— Check reminder_count — if >= FOLLOWUP_LIMITS.maxReminders → mark stale, stop
|
|— Send Resend email using buildFollowupEmail():
|   "Hey [name] — you mentioned you'd get [item] for [startup] by [date]."
|   CTA: Update in portal → link to /scout
|
|— Increment reminder_count, update last_reminded_at
```

4.5 Market Analysis Agent

Lives at `src/app/api/internal/deals/[dealId]/analyze/route.ts` . Triggered on-demand by Quanta team.

```
POST /api/internal/deals/:id/analyze
|
|— Assemble deal context (name, desc, category, stage, signals, conversation
excerpts)
|
|— Run market analysis
|   Model: llama-3.3-70b-versatile (Groq)
|   Output: {
|     market_overview, market_size (TAM/SAM + note),
|     tailwinds[], headwinds[],
|     comparable_companies[] (name, similarity, differentiation, outcome),
|     investment_thesis, key_diligence_questions[],
|     verdict (promising/neutral/concerning) + verdict_reason,
|     visualizations[] (AI-chosen chart types + data)
|   }
|
|— Cached in ai_outputs (output_type = 'enrichment')
   GET endpoint returns cached result (with generated_at timestamp)
```

4.6 Prompt Architecture

Every prompt is isolated in `src/prompts/` . Each file exports:

- `<NAME>_SYSTEM_PROMPT` — the static system message
- `build<Name>UserPrompt(...)` — assembles the dynamic user message from context

This separation ensures:

- Prompts can be iterated without touching agent code
- Prompts are testable in isolation
- No prompt strings are scattered in API routes or agent files

5. OpenClaw — The Messaging Gateway

What OpenClaw Is (and Isn't)

OpenClaw is a **self-hosted multi-channel messaging gateway**. It sits between messaging platforms (Telegram, WhatsApp, Slack, SMS) and our backend. It handles protocol translation, session management, and delivery routing — so our backend receives a normalized payload regardless of which channel the scout used.

In the current build: OpenClaw is architecturally integrated but operationally inactive. The code for the webhook (`/api/openclaw/webhook`) and the sender (`src/lib/openclaw/client.ts`) exist and are tested, but messages are routed through the web app instead because:

1. OpenClaw requires a self-hosted server (not zero-cost)
2. Telegram Bot API and WhatsApp Business API require verified business accounts for production use
3. The web app covers the full feature set needed for the demo

In a production deployment: OpenClaw would be the primary scout input channel. Scouts would never visit a website — they'd text the Quanta Telegram bot.

How the Webhook Works



```
deal_id: "abc-123"
}
```



OpenClaw delivers reply to Telegram

Sending Proactive Messages via OpenClaw

For check-ins and follow-ups, the system calls `sendMessage()` from `src/lib/openclaw/client.ts` :

```
await sendMessage({
  openclaw_user_id: "tg_123456",
  channel: "telegram",
  message: "Seen any interesting founders this week?"
});
```

When `OPENCLAW_SIMULATE=true` is set in env (current state), this logs to console instead of sending. In production, this would deliver to Telegram.

Channel Capability Matrix

Channel	Text	Voice Notes	File Upload	Reactions	CTA Buttons
Telegram	✓	✓	✓	✓	✓ (inline keyboards)
WhatsApp	✓	✓	✓	Limited	✓ (template messages)
Slack	✓	✗	✓	✓	✓ (Block Kit)
SMS	✓	✗	✗	✗	✗
Web (current)	✓	✓	✓	✗	✓

6. Telegram Bot as the Interaction Layer

Why Telegram Replaces the Web Interaction Tab

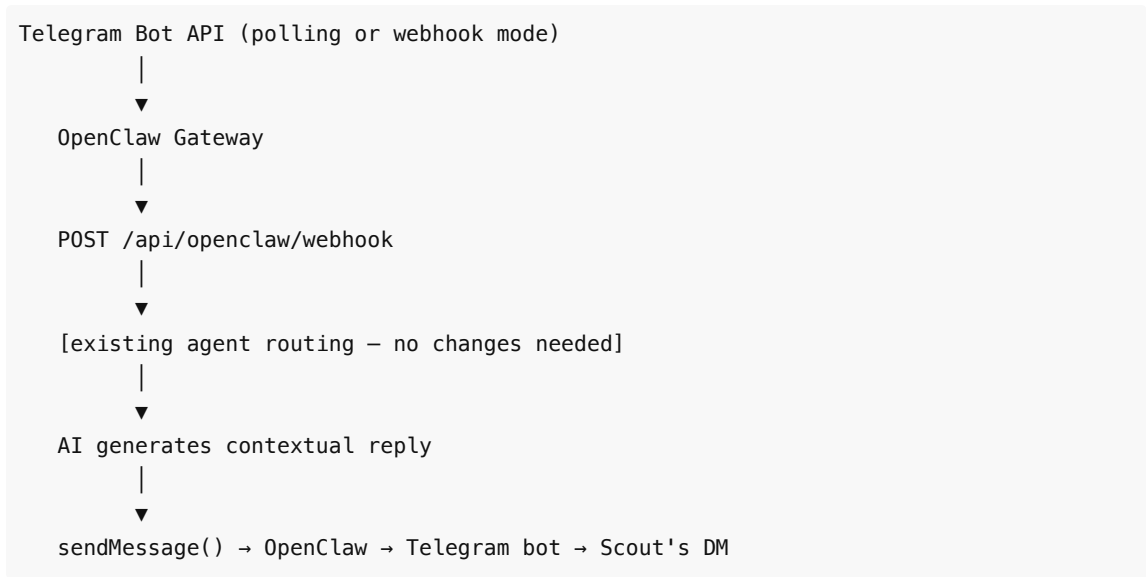
The web-based Interaction tab is a compromise. Scouts visit a portal, find their deal, open a tab, and chat. This adds 4+ steps of friction before a scout can update Quanta on anything.

With a Telegram bot, the interaction becomes:

```
Scout on Telegram: "hey, HealthThread got their first paying hospital contract"
Quanta bot:        "That's a strong signal. What's the contract value – annual or
monthly?"
Scout:             "₹12L annually, 2-year lock-in"
Quanta bot:        "Perfect. I've updated the deal. We'll flag this for the team."
```

No app. No login. No "open the portal". The scout just texts.

Technical Implementation



The **entire backend already supports this** — the webhook, agent routing, and sendMessage() are all built. What's needed to activate it:

- 1. Create Telegram bot via @BotFather → get TELEGRAM_BOT_TOKEN
- 2. Deploy OpenClaw or use a Telegram-native webhook (simpler)
- 3. Set OPENCLAW_SIMULATE=false and configure the bot token
- 4. Point the bot's webhook URL to /api/openclaw/webhook

Telegram-Specific Enhancements

Once Telegram is live:

Inline keyboards for yes/no: Instead of a text reply, the bot sends buttons:

```
Have you raised any external funding?
[Yes, I have] [Not yet] [Looking to raise now]
```

Voice note handling: Scouts record voice notes directly in Telegram. OpenClaw downloads the audio, sends it to our /api/startup/:id/audio endpoint, transcribes, and responds with extracted data for confirmation.

Persistent menu commands:

```
/submit – Start a new deal submission
/status – Check status of your latest deals
/update [deal name] – Send an update on a specific deal
```

Group handling: If a scout adds the bot to a group chat, it can listen for deal-related messages in context, useful for "office WhatsApp" scenarios where founders are discussed informally.

7. Current Tools & Free-Tier Constraints

Active Stack (what's running now)

Tool	Purpose	Free Tier	Constraint
------	---------	-----------	------------

Vercel (Hobby)	Hosting + cron	Free	10s function timeout; 100GB bandwidth
Supabase (Free)	Postgres + Auth + Storage	Free	500MB DB; 1GB Storage; 2 projects
Groq	LLM completions + Whisper STT	Free	28,800 audio sec/day; token rate limits
Resend	Transactional email	Free (100/day)	Unverified domain → can only send to account owner's email
OpenClaw	Messaging gateway	—	Not active — simulated via <code>OPENCLAW_SIMULATE=true</code>
Vercel Cron	Weekly check-in scheduler	Free (1 cron/project)	1 cron job on free plan

Current Operational Gaps

Email: With no verified domain, all outbound emails only deliver to `atharvanimbalkar36@gmail.com`. To send to any scout, Resend needs a verified domain (`scout@quantaventures.com` or similar).

OpenClaw / Telegram: The integration is built and tested but not active. Scouts use the web portal instead of texting. This is the biggest gap between current state and the intended UX.

Function timeout: Vercel Hobby plan has a 10-second function timeout. The AI pipeline (7 steps, multiple Groq calls) runs in the background so it doesn't block the user. But complex market analysis requests can approach the limit.

Audio storage: Voice pitches are stored in Supabase Storage. The free tier gives 1GB — enough for hundreds of demos but not production scale.

8. Production Upgrades — What Changes and Why

Messaging Infrastructure

Current	Production	Why
Web portal (scout types here)	Telegram bot + web as fallback	Scouts don't want another app. They'll ghost a portal but text a bot.
<code>OPENCLAW_SIMULATE=true</code>	Full OpenClaw deployment	Real channels: Telegram, WhatsApp, Slack
Resend shared domain	Verified domain (e.g., <code>scout@quantaventures.com</code>)	Email can reach any recipient, not just the registered Resend account

Compute & Reliability

Current	Production	Why
---------	------------	-----

Vercel Hobby (10s timeout)	Vercel Pro (60s timeout)	Complex AI calls need headroom
Fire-and-forget pipeline	Inngest background jobs	Retries, observability, failure alerts — pipeline failures are invisible now
No rate limiting	Upstash Redis edge rate limiting	Prevent accidental or malicious spam submissions

AI & Intelligence

Current	Production	Why
Document-only enrichment	Live web research (Exa/Perplexity API)	Auto-enrich every deal without scout effort
String-match duplicate detection	pgvector semantic embeddings	Catches "FlowOps" vs "Flow Operations" — same company, different name
No founder verification	LinkedIn auto-fetch (Proxycurl API)	Background, education, previous companies pulled automatically
Groq Whisper (28,800 sec/day shared)	AssemblyAI or dedicated Whisper	Unlimited audio, speaker diarization, chapter markers

Database & Search

Current	Production	Why
No search	pgvector semantic search	"Find deals similar to AgriPulse" returns semantically similar, not keyword matches
Manual deal comparison	AI-powered comps intelligence	Auto-surface comparable companies for every new deal
No portfolio conflict check	Automated overlap detection	Flag when new deal competes with existing portfolio company

9. AI Model Choices — Groq vs OpenAI

Why Groq Is Used Now

Groq's LPU (Language Processing Unit) hardware provides inference speeds 5-10x faster than GPU-based providers. For a real-time scout chat experience, response latency matters. Groq's free tier is also generous:

- `llama-3.3-70b-versatile` — high-quality reasoning, structured JSON output
- `llama-3.1-8b-instant` — fast classification tasks (review labels, commitment detection)
- `whisper-large-v3-turbo` — fastest Whisper variant, 5-10x faster than OpenAI's

The Groq Limitations

Limitation	Impact
------------	--------

No embeddings API	Can't do semantic search or duplicate detection by similarity
Rate limits on free tier	Burst usage (many scouts submitting simultaneously) can hit limits
Model availability	Llama models are generally capable but lag behind GPT-4o on nuanced reasoning
No vision/multimodal	Can't process images directly (only text extracted from PDFs)

Should You Use OpenAI Instead?

Short answer: Not entirely — a hybrid approach is better.

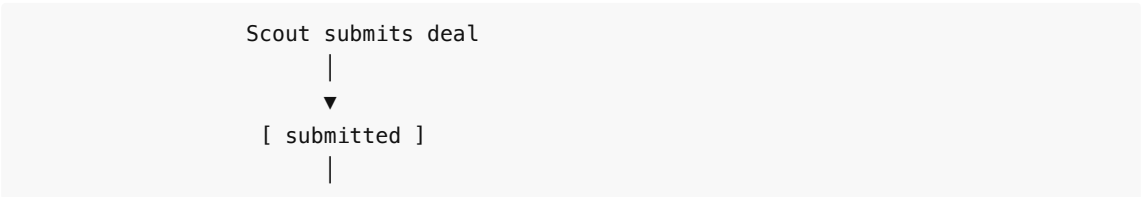
Task	Best model	Why
High-precision structured extraction	GPT-4o	Better instruction following, fewer hallucinations on edge cases
Real-time chat replies (scout liaison)	Groq Llama	Speed > marginal quality improvement
Classification tasks (review label, commitment)	Groq Llama 8b	Fast, cheap, accurate enough for binary classification
Market analysis (investment thesis, comps)	GPT-4o	Deeper reasoning and knowledge for nuanced VC analysis
Speech-to-text	Groq Whisper	5-10x faster than OpenAI Whisper at similar accuracy
Semantic embeddings	OpenAI text-embedding-3-small	Groq doesn't offer embeddings at all

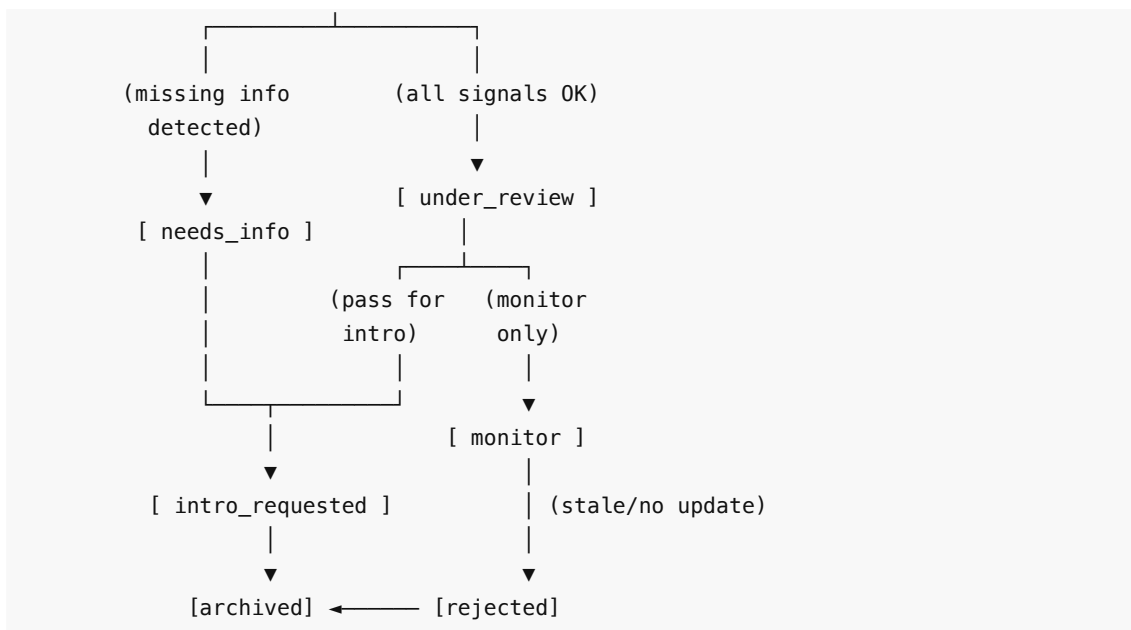
Recommended Production Split

Groq Whisper	→ all voice transcription
Groq Llama 3.1-8b	→ commitment detection, review label, follow-up messages
Groq Llama 3.3-70b	→ real-time chat replies, check-in messages
GPT-4o	→ extraction (new deal intake), internal brief, market analysis
OpenAI Embeddings	→ semantic duplicate detection, deal search

This hybrid reduces costs (Groq is free / cheap for fast tasks), improves quality on high-stakes decisions (GPT-4o for briefs and analysis), and keeps latency low for the scout experience.

10. Deal State Machine





Status definitions:

- `temp` — deal started but not yet submitted (discarded on abandon)
- `draft` — partially completed, saved but not sent to Quanta
- `submitted` — scout has submitted, pipeline running
- `needs_info` — one or more `missing_info_tasks` pending
- `under_review` — Quanta team is actively reviewing
- `intro_requested` — Quanta wants an introduction to the founder
- `monitor` — worth watching but not ready to move forward
- `archived` / `rejected` — terminal states

11. Limitations & Known Constraints

Technical

Limitation	Current workaround	Real fix
Vercel 10s function timeout	Pipeline runs in background (non-blocking)	Vercel Pro or Inngest for durable jobs
No real-time updates	Page re-fetches on <code>visibilitychange</code>	Supabase Realtime WebSockets
Next.js Data Cache caches Supabase SDK <code>fetch()</code> calls	Direct <code>fetch()</code> with cache: 'no-store' for messages	Inngest + webhooks so DB changes push to client
Email only reaches one address (no verified domain)	Demo use only	Verify <code>scout@quantaventures.com</code> on Resend
OpenClaw is simulated	Web portal as substitute	Deploy OpenClaw + configure Telegram bot

No semantic duplicate detection	String similarity	pgvector + OpenAI embeddings
Voice recordings stored in Supabase (1GB free limit)	Works for demo	Cloudflare R2 or S3 for production

Product

Limitation	Impact
Scout must use web portal (no Telegram)	Higher drop-off rate — scouts prefer messaging apps
No push notifications	Scout doesn't know Quanta asked a question unless they check the portal
Market analysis is manual (click-to-generate)	Should auto-trigger after signals are generated
No portfolio conflict detection	Risk of sending deals that compete with portfolio companies
Follow-up emails go to one mailbox (Resend free)	Cannot test multi-scout email flow

AI

Limitation	Impact
Extraction can hallucinate stage/fundraising if scout is vague	Low-confidence extractions need human review
Chat liaison has no memory beyond 30 messages	Very long conversations lose early context
Market analysis is AI knowledge-cutoff bound	TAM/SAM estimates are not live — stated clearly in UI
Voice pitch extracts all fields in one pass	If pitch is unclear, all fields get low-confidence values

12. Future Scope

Near Term (1-3 months)

Telegram bot activation The biggest UX improvement available. All backend code exists. Just needs:

- Telegram bot registration (@BotFather)
- OpenClaw deployment or direct webhook configuration
- Set `OPENCLAW_SIMULATE=false`

Verified email domain Register `quantaventures.com` or similar on Resend. Unlocks multi-scout email delivery. Cost: ~\$15/year for the domain.

Vercel Pro Increases function timeout to 60 seconds, removes bandwidth caps. Needed before any real scout load. Cost: \$20/month.

Inngest background jobs Replace fire-and-forget `runPostSubmissionPipeline()` with durable, retryable jobs. When a step fails, it retries automatically and sends an alert. Cost: free tier is generous.

Medium Term (3-6 months)

Live web enrichment (Exa or Perplexity) Every deal auto-researched after submission. Pulls LinkedIn profiles, Crunchbase funding history, press coverage, domain traffic estimates. No extra work from the scout.

pgvector semantic duplicate detection Embeds every deal at submission using OpenAI text-embedding-3-small. Cosine similarity search against all existing deals. Catches "FlowOps" and "Flow Operations" as the same company.

Supabase Realtime Chat messages update live across scout and team views without polling or page refreshes. Presence indicators ("Evan is viewing this deal") for Quanta team.

Partner weekly digest AI-generated Monday morning email to Evan and Mateo: new deals this week, follow-ups due, status changes, suggested actions. Formatted like an internal investment memo.

Long Term (6-12 months)

React Native mobile app Native iOS and Android app for scouts. Native audio recording (better quality than browser), offline submissions that sync on connectivity, push notifications, camera for pitch deck photos.

WhatsApp Business API Full WhatsApp integration via Meta's official API. Required for scouts in India who prefer WhatsApp over Telegram. Requires Meta Business Verification.

GPT-4o Vision Scout photographs a slide deck or whiteboard → system reads and structures it directly. Handles physical decks at events.

Portfolio conflict detection Automatically checks each new deal against existing portfolio companies for overlap. Flags before the team reviews.

Quanta CoPilot sidebar AI assistant on the deal detail page that answers questions using your full deal history + live web search. "How does this compare to our logistics bets?" "What's the relevant market size for agri-fintech in Maharashtra?"

Multi-fund / white-label The system architecture is generic — it's a scout intelligence layer that works for any investment firm. Parameterise the fund name, brand, thesis, and deploy separately for each client.